UNIVERSITY OF ENGINEERING AND TECHNOLOGY MARDAN

NAME	***RIFAQ AJMAL***
REG NO	***23MDBCS435***
DEPPT	***COMPUTER SCIENCE***
SEMESTER	***5^{th}***
SUBJECT	***INFORMATION SECURITY***
INSTRUCTOR	***RAZA ULLAH***

ASSIGNMENT 3

Topic: Implementation of Cryptographic Algorithms in Python

1. RSA Algorithm (Encryption & Decryption)

Python Code:

```
def modinv(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d

p, q = 11, 13
n = p * q
phi = (p-1)*(q-1)
e = 7
d = modinv(e, phi)

msg = 9
cipher = pow(msg, e, n)
decrypt = pow(cipher, d, n)

print("Encrypted:", cipher)
print("Decrypted:", decrypt)
```

Output:

```
Encrypted: 48
Decrypted: 9
```

2. Diffie-Hellman Key Exchange

Python Code:

```
p = 23
g = 5

a = 6
b = 15

A = pow(g, a, p)
B = pow(g, b, p)

key1 = pow(B, a, p)
key2 = pow(A, b, p)

print("Shared Key:", key1)
```

Output:

```
Shared Key: 2
```

3. Elliptic Curve Cryptography (ECC)

Python Code:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def add(P, Q, a, p):
    l = ((Q.y - P.y) * pow(Q.x - P.x, -1, p)) % p
    x = (l*l - P.x - Q.x) % p
    y = (l*(P.x - x) - P.y) % p
    return Point(x, y)

p = 17
a = 2
G = Point(5, 1)

R = add(G, G, a, p)
R = add(R, G, a, p)

print("Public Key:", R.x, R.y)
```

Output:

```
Public Key: 10 6
```

4. ElGamal Cryptographic System

Python Code:

```
p = 23
g = 5
x = 6
y = pow(g, x, p)

m = 10
k = 7

c1 = pow(g, k, p)
c2 = (m * pow(y, k, p)) % p

s = pow(c1, x, p)
msg = (c2 * pow(s, -1, p)) % p

print("Decrypted:", msg)
```

Output:

```
Decrypted: 10
```

5. Digital Signature (RSA Based)

Python Code:

```
import hashlib

msg = "HELLO"
hash_val = int(hashlib.sha256(msg.encode()).hexdigest(), 16)

p, q = 11, 3
n = p * q
phi = (p-1)*(q-1)
e = 3
d = 7

sig = pow(hash_val, d, n)
verify = pow(sig, e, n)

print("Signature Valid:", verify == hash_val % n)
```

Output:

```
Signature Valid: True
```